

Solutionnaire du TP1

À votre tour

0. Importer les données

```
library(tidyverse)
grades <- read_tsv("/home/public/tidyverse/grades.tsv")
```

```
#> Rows: 79 Columns: 9
#> — Column specification —————
#> Delimiter: "\t"
#> chr (2): ID_etu, Cohorte
#> dbl (7): D1, D2, E1, D3, D4, E2, Annee
#>
#> i Use `spec()` to retrieve the full column specification for this data.
#> i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

79 lignes et 9 colonnes. **ID_etu** et **Cohorte** sont du texte (**chr**), les 6 notes et **Annee** sont numériques (**dbl**).
On vérifie avec :

```
glimpse(grades)
#> Rows: 79
#> Columns: 9
#> $ ID_etu   <chr> "Etu_1", "Etu_2", "Etu_3", "Etu_4", "Etu_5", "Etu_6", "Etu_7", ...
#> $ D1       <dbl> 90.00, 86.67, 83.33, 83.33, 86.67, 100.00, 70.00, 96.67, 63.33...
#> $ D2       <dbl> 87.5, 88.0, 70.0, 85.0, 70.0, 85.0, 13.8, 92.5, 85.0, 68.8, 83...
#> $ E1       <dbl> 94.44, 83.33, 56.67, 77.78, 40.00, 76.67, 51.11, 90.00, 85.56, ...
#> $ D3       <dbl> 100.0, 100.0, 80.0, 100.0, 80.0, 100.0, 55.0, 100.0, 90.0, 100...
#> $ D4       <dbl> 98.0, 98.0, 65.0, 100.0, 65.5, 100.0, 0.0, 90.0, 100.0, 100.0, ...
#> $ E2       <dbl> 92.86, 90.00, 30.71, 82.86, 5.71, 34.29, 48.57, 76.43, 81.43, ...
#> $ Annee    <dbl> 2018, 2018, 2018, 2018, 2018, 2018, 2018, 2018, 2018, 2018, 20...
#> $ Cohorte  <chr> "Cycle 1", "Cycle 1", "Cycle 1", "Cycle 1", "Cycle 1", "Cycle 1", "Cycle ...
```

1. Choisir des colonnes

```
grades |> select(ID_etu, E1, E2)
#> # A tibble: 79 × 3
#>   ID_etu     E1     E2
```

```
#>   <chr>   <dbl> <dbl>
#> 1 Etu_1    94.4 92.9
#> 2 Etu_2    83.3 90
#> 3 Etu_3    56.7 30.7
#> 4 Etu_4    77.8 82.9
#> 5 Etu_5     40   5.71
#> 6 Etu_6    76.7 34.3
#> 7 Etu_7    51.1 48.6
#> 8 Etu_8     90   76.4
#> 9 Etu_9    85.6 81.4
#> 10 Etu_10  54.4 25.7
#> # i 69 more rows
```

2. Les étudiants de 2019

```
grades |> filter(Annee == 2019)
#> # A tibble: 26 × 9
#>   ID_etu    D1    D2    E1    D3    D4    E2 Année Cohorte
#>   <chr>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
#> 1 Etu_31  70      0  61.7    60    0   54.3  2019 Cycle 1
#> 2 Etu_32  90      90  68.3    75  62.5  71.4  2019 Cycle 1
#> 3 Etu_33  96.7    80  72.2   100  95   68.6  2019 Cycle 1
#> 4 Etu_34  93.3    55  66.1   100  85   72.9  2019 Cycle 1
#> 5 Etu_35  100     80  67.2   100  80   64.3  2019 Cycle 1
#> 6 Etu_36  90      90  62.2    75  64.5  58.6  2019 Cycle 1
#> 7 Etu_37  76.7    75  68.9   100  90   57.9  2019 Cycle 1
#> 8 Etu_38  100    100  87.2   100  95   93.6  2019 Cycle 1
#> 9 Etu_39  96.7   100  61.7   100  95   67.9  2019 Cycle 1
#> 10 Etu_40  93.3    55  58.9    95  65    65   2019 Cycle 1
#> # i 16 more rows
grades |> filter(Annee == 2019) |> nrow()
#> [1] 26
```

3. 2019 et Cycle 1

Dans `filter()`, la virgule vaut « et » (&) :

```
grades |> filter(Annee == 2019, Cohorte == "Cycle 1") |> nrow()
#> [1] 14
# équivalent : filter(Annee == 2019 & Cohorte == "Cycle 1")
```

4. Ajouter la moyenne des examens

```
grades |>
  mutate(moyenne_exam = (E1 + E2) / 2) |>
  select(ID_etu, E1, E2, moyenne_exam)
#> # A tibble: 79 × 4
```

```
#>   ID_etu    E1    E2 moyenne_exam
#>   <chr>  <dbl> <dbl>         <dbl>
#> 1 Etu_1   94.4 92.9           93.6
#> 2 Etu_2   83.3 90            86.7
#> 3 Etu_3   56.7 30.7           43.7
#> 4 Etu_4   77.8 82.9           80.3
#> 5 Etu_5    40  5.71           22.9
#> 6 Etu_6   76.7 34.3           55.5
#> 7 Etu_7   51.1 48.6           49.8
#> 8 Etu_8    90  76.4           83.2
#> 9 Etu_9   85.6 81.4           83.5
#> 10 Etu_10  54.4 25.7           40.1
#> # i 69 more rows
```

5. Plus de 90 à l'examen E2

```
grades |> filter(E2 > 90) |> nrow()
#> [1] 21
# autre façon, sans filtrer : TRUE vaut 1 et FALSE vaut 0
sum(grades$E2 > 90)
#> [1] 21
```

6. Les 5 meilleures notes à E2

```
grades |>
  arrange(desc(E2)) |>
  select(ID_etu, E2) |>
  head(5)
#> # A tibble: 5 × 2
#>   ID_etu    E2
#>   <chr>  <dbl>
#> 1 Etu_54  98.6
#> 2 Etu_68  98.5
#> 3 Etu_45  97.9
#> 4 Etu_52  97.9
#> 5 Etu_44  97.1
```

`slice_max(E2, n = 5)` donne le même résultat en une étape (et garde les ex aequo).

7. Nombre d'étudiants par cohorte et par année

```
grades |> count(Cohorte)
#> # A tibble: 2 × 2
#>   Cohorte      n
#>   <chr>    <int>
#> 1 Cycle 1      41
#> 2 Cycle 2 et 3  38
```

```
grades |> count(Annee)
#> # A tibble: 3 × 2
#>   Annee     n
#>   <dbl> <int>
#> 1  2018     30
#> 2  2019     26
#> 3  2020     23
```

Et si... ?

1. `Annee == 2019 | 2020`

```
grades |> filter(Annee == 2019 | 2020) |> nrow()
#> [1] 79
```

Les **79** étudiants. R lit `(Annee == 2019) | 2020` : le nombre `2020` est converti en logique, et tout nombre non nul vaut **TRUE**. La condition est donc toujours vraie, sans aucun message. Il faut répéter la colonne, ou utiliser `%in%` :

```
grades |> filter(Annee == 2019 | Annee == 2020) |> nrow()
#> [1] 49
grades |> filter(Annee %in% c(2019, 2020)) |> nrow()
#> [1] 49
```

2. `Cohorte == c("Cycle 1", "Cycle 2 et 3")`

```
grades |> filter(Cohorte == c("Cycle 1", "Cycle 2 et 3")) |> nrow()
#> Warning: There was 1 warning in `filter()`.
#> i In argument: `Cohorte == c("Cycle 1", "Cycle 2 et 3")`.
#> Caused by warning in `Cohorte == c("Cycle 1", "Cycle 2 et 3")`:
#> ! longer object length is not a multiple of shorter object length
#> [1] 40
```

Non : `==` compare **élément par élément** et recycle le vecteur court. La ligne 1 est comparée à **"Cycle 1"**, la ligne 2 à **"Cycle 2 et 3"**, la ligne 3 à **"Cycle 1"** ... On garde environ la moitié des étudiants, au hasard de leur position. Ici, 79 n'est pas un multiple de 2 : R affiche un *warning*. Avec un nombre pair de lignes, il n'y aurait **aucun message**. La bonne écriture :

```
grades |> filter(Cohorte %in% c("Cycle 1", "Cycle 2 et 3")) |> nrow()
#> [1] 79
```

3. Moins de 70 ou 70 et plus au devoir D2

```

grades |> filter(D2 < 70) |> nrow()
#> [1] 7
grades |> filter(D2 >= 70) |> nrow()
#> [1] 71
grades |> filter(is.na(D2))
#> # A tibble: 1 × 9
#>   ID_etu    D1    D2    E1    D3    D4    E2 Annee Cohorte
#>   <chr>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
#> 1 Etu_63    60    NA  76.4    50  97.1  82.1  2020 Cycle 1

```

La somme fait 78 : l'étudiant **Etu_63** n'a pas de note au devoir 2 (**NA**). Pour lui, **NA < 70** et **NA >= 70** valent tous les deux **NA**, et **filter()** écarte les lignes où la condition n'est pas **TRUE**. Il disparaît **des deux** sélections sans message. Vérification utile : la somme des parties doit faire le tout.

4. Oublier d'assigner

```

grades |> mutate(moyenne_exam = (E1 + E2) / 2) |> head(2)
#> # A tibble: 2 × 10
#>   ID_etu    D1    D2    E1    D3    D4    E2 Annee Cohorte moyenne_exam
#>   <chr>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>          <dbl>
#> 1 Etu_1    90   87.5  94.4  100    98  92.9  2018 Cycle 1          93.6
#> 2 Etu_2   86.7  88    83.3  100    98  90    2018 Cycle 1          86.7
grades$moyenne_exam
#> Warning: Unknown or uninitialised column: `moyenne_exam`.
#> NULL

```

mutate() renvoie un **nouveau** tableau, qui est affiché puis perdu : **grades** n'a pas changé. Le *tibble* avertit que la colonne n'existe pas et renvoie **NULL**. Pour conserver la colonne, il faut assigner le résultat :

```

grades <- grades |> mutate(moyenne_exam = (E1 + E2) / 2)

```

5. Trier avec un NA

```

grades |> arrange(D2) |> select(ID_etu, D2) |> tail(2)
#> # A tibble: 2 × 2
#>   ID_etu    D2
#>   <chr>  <dbl>
#> 1 Etu_78  100
#> 2 Etu_63   NA
grades |> arrange(desc(D2)) |> select(ID_etu, D2) |> tail(2)
#> # A tibble: 2 × 2
#>   ID_etu    D2
#>   <chr>  <dbl>
#> 1 Etu_31    0
#> 2 Etu_63   NA

```

Dans les deux cas, `Etu_63` est **à la fin** : `arrange()` place toujours les `NA` en dernier, quel que soit le sens du tri.

